

PROJETO 2: PRIMEIROS PASSOS PARA O PROJETO DE UM SO

Izaque Vieira de Souza¹, Olávio Gonçalves de Almeida¹, Wisley Kilder da Silva Nogueira¹

¹Departamento de Ciência da Computação – Universidade Federal de Goiás (UFG)
izaquevieira@outlook.com.br, olaviog.almeida@gmail.com,
w.kilder@hotmail.com

1. Tarefa 1

1.1. Descrição

A primeira tarefa consiste nos passos básicos para a criação de um SO. Nessa tarefa será feita a preparação do sistema, bem como instalação de alguns *softwares*. Serão apresentadas também algumas dificuldades encontradas no decorrer do Projeto 2.

1.2. Instalação de *softwares* necessários

Foram usados comandos separados para a instalação de cada *softwares* que foi pedido. Isso aconteceu porque alguns dos *softwares* já estavam instalados na máquina, sendo assim seria desnecessário repetir o comando para a instalação do mesmo. Os comandos utilizados para a instalação dos *softwares* foram:

```
sudo apt-get update  
sudo apt-get install xorriso  
sudo apt-get install nasm  
sudo apt-get install qemu
```

Alguns *softwares* como o *grub* e o *gcc* já estavam instalados na máquina.

A instalação do VirtualBox por meio dos repositórios padrões do sistema Linux não foi possível, sendo necessário buscar ajuda em um blog (CASTRO, 2018). Com essa ajuda foi possível utilizar a ferramenta VirtualBox.

Cada *software* instalado acima possui uma finalidade:

- xorriso: gerar a imagem do sistema (arquivo .iso);
- nasm: compilador Assembly, utilizado em para editar pontos de entrada do *kernel*;
- grub: utilizado para criar o *bootloader*;
- gcc: utilizado para compilar os códigos C;

- qemu: execução direta do arquivo .bin para testar o SO criado, evitando gerar uma imagem do sistema a cada alteração feita;
- VirtualBox: utilizado para testar a imagem ISO gerada do sistema.

1.3. Criação dos primeiros arquivos

Para prosseguir, foi criado um diretório chamado “SO”, e dentro deste diretório foram criados os arquivos *kernel.asm*, *kernel.c*, *link.ld* e *build.sh*. A implementação de cada código para os arquivos descritos acima foi feita com base no material disponibilizado pelo professor.

O código presente no arquivo *kernel.asm* é responsável por criar o ponto de entrada do *kernel*. A Imagem 1 representa o código presente em *kernel.asm*:

```

1 bits 32
2 section          .text
3                 align 4
4                 dd  0x1BADB002
5                 dd  0x00
6                 dd  - (0x1BADB002+0x00)
7 global          start
8 extern          kmain
9 start:
10                cli
11                call kmain
12                hlt
--
```

Imagem 1: Código de *kernel.asm*

Na imagem, a linha 1 quer dizer ao *nasm* que ele deve gerar um código a ser executado em um processador de 32 bits. Para as linhas de 2 até 6 são especificações para o *grub* e *bootloader*. Na linha 7, é criada uma função *start*, de modo global, para que a mesma seja chamada no arquivo *link.ld*. A função *kmain* faz referência à função que deve ser criada no arquivo *kernel.c*.

O código inserido no arquivo *link.ld* foi fornecido no material, contendo o seguinte conteúdo:

```

1 OUTPUT_FORMAT(elf32-i386)
2 ENTRY(start)
3 SECTIONS
4 {
5     . = 0x100000;
6     .text : { *(.text) }
7     .data : { *(.data) }
8     .bss : { *(.bss) }
9 }
```

Imagem 2: Código de *link.ld*

O arquivo *link.ld* será utilizado para juntar os arquivos gerados ao compilar o arquivo *Assembly* e ao compilar o código *kernel.c*, gerando assim um arquivo com extensão *.bin*.

O documento *kernel.c* possui o código apresentado abaixo na Imagem 3. Este por sua vez, possui a função *kmain()*, onde o objetivo dela até o presente momento é mostrar um caractere da cor verde na tela.

```
6 kmain(){
7
8     char *vidmem = (char*) 0xb8000;
9     vidmem[0] = 'X';
10    vidmem[1] = 0x0F; /* cor do caracter: verde */
11 }
```

Imagem 3: Código de *kernel.c*

Para compilar os arquivos criados até o presente momento, foi organizado um arquivo *shell script*, contendo os comandos para compilar os arquivos *kernel.asm*, *kernel.c*, e posteriormente, contém o comando para fazer a junção através do *link.ld* dos arquivos gerados pela compilação dos códigos anteriores. Quando essa junção é feita, um arquivo *kernel.bin* é gerado, e a partir dele, podemos usar o *qemu* para fazer um rápido teste do sistema criado até o momento, sem precisar gerar uma imagem do sistema. Segue abaixo na Imagem 4 o código do *shell script build.sh*:

```
1 #!/bin/sh
2 nasm -f elf32 kernel.asm -o kasm.o
3 gcc -m32 -c -w kernel.c -o kc.o
4 ld -m elf_i386 -T link.ld -o kernel.bin kasm.o kc.o
5 qemu-system-i386 --kernel kernel.bin
6 mv kernel.bin simple/boot/
```

Imagem 4: Código de *build.sh*

Nota-se que foram adicionados os comandos da linha 5 e 6 no arquivo *build.sh*. O comando apresentado na linha 5 é usado para que o *qemu* já execute o arquivo *kernel.bin* assim que o arquivo *build.sh* for executado, e o comando da linha 6 moverá o arquivo *kernel.bin* gerado para o diretório indicado, facilitando ao gerar a imagem do sistema posteriormente. Por fim, para dar permissão de execução ao *shell script* criado, o seguinte comando foi utilizado:

```
sudo chmod +777 build.sh
```

Ao executar o arquivo *shell script* criado, o resultado é apresentado na Imagem 5.

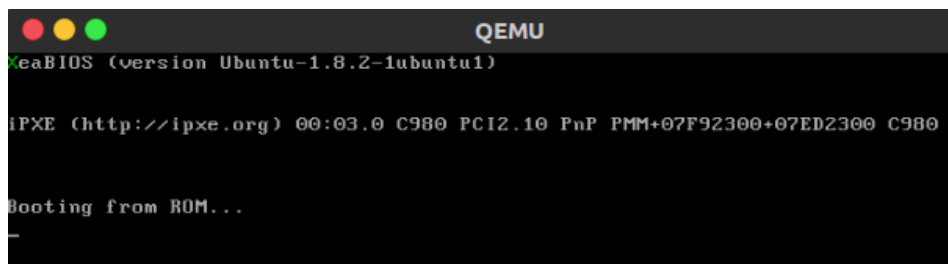


Imagem 5: Resultado da primeira execução

Para a criar uma ISO do sistema feito até então, foi necessário criar alguns diretórios dentro do diretório *SO*, resultando na seguinte ordem: *SO/simple/boot/grub/*. Dentro do diretório *boot* está localizado o arquivo *kernel.bin*, e dentro do diretório *grub*, será criado um arquivo que fará a configuração do *grub*. No código apresentado abaixo na Imagem 6, as configurações são feitas de modo que o tempo aguardando a inicialização do sistema seja 0 segundos (`set timeout = 0`), para que seu *boot* seja feito através do disco óptico (`set root = '(hd96)'` e também, para que seu nome seja “simpleOS”.

```
1 set default = 0
2 set timeout = 0
3
4 menuentry "simpleOS" {
5     set root='(hd96)'
6     multiboot /boot/kernel.bin
7 }
```

Imagem 6: Código do arquivo *grub.cfg*

Para gerar a ISO do sistema criado, o seguinte comando foi utilizado:

```
grub-mkrescue -o simple.iso simple
```

Ao testar a nova ISO criada no VirtualBox, escrevendo uma palavra na tela, o resultado foi o seguinte (Imagem 7):

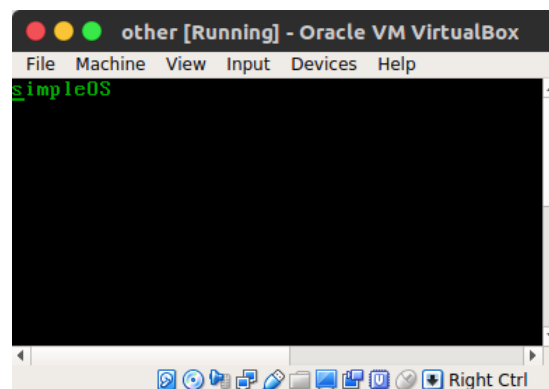


Imagem 7: Execução da ISO criada através do VirtualBox

Para escrever a frase da imagem acima (Imagem 7), o código do arquivo *kernel.c* foi alterado. O mesmo é apresentado na Imagem 8:

```

1 |kmain(){
2 |    char *vidmem = (char*) 0xb8000;
3 |    vidmem[0] = 's';
4 |    vidmem[1] = 0x02; /* cor do character: verde */
5 |
6 |    vidmem[2] = 'i';
7 |    vidmem[3] = 0x02;
8 |
9 |    vidmem[4] = 'm';
10 |    vidmem[5] = 0x02;
11 |
12 |    vidmem[6] = 'p';
13 |    vidmem[7] = 0x02;
14 |
15 |    vidmem[8] = 'l';
16 |    vidmem[9] = 0x02;
17 |
18 |    vidmem[10] = 'e';
19 |    vidmem[11] = 0x02;
20 |
21 |    vidmem[12] = 'o';
22 |    vidmem[13] = 0x02;
23 |
24 |    vidmem[14] = 'S';
25 |    vidmem[15] = 0x02;
26 |}

```

Imagem 8: Código de *kernel.c* para escrever “simpleOS”

2. Tarefa 2

2.1. Criando os arquivos

Os arquivos *types.h*, *system.h*, *string.h* e *screen.h* foram criados dentro do novo diretório chamado *include*, conforme dito no documento de orientação.

2.1.1. Arquivo *types.h*

Neste arquivo foram inseridos os códigos necessários para a criação dos tipos primitivos de dados que serão usados pelas funções do SO. O código é apresentado na Imagem 9:

```

1 |#ifndef TYPES_H
2 |#define TYPES_H
3 |
4 |/* As funções abaixo são usadas
5 |para definir os tipos de dados
6 |que serão utilizados no decorrer
7 |das tarefas*/
8 |
9 |typedef signed char int8;
10 |typedef unsigned char uint8;
11 |
12 |typedef signed short int16;
13 |typedef unsigned short uint16;
14 |
15 |typedef signed int int32;
16 |typedef unsigned int uint32;
17 |
18 |typedef signed long int64;
19 |typedef unsigned long uint64;
20 |
21 |typedef char* string;
22 |
23 |#endif

```

Imagem 9: Código do arquivo *types.h*

2.1.2. Arquivo *string.h*

1) O que a função realiza é a contagem de quantos caracteres possui na *string*, ou seja, o tamanho da *string* que foi passada para a função como parâmetro. Todos esses procedimentos são feitos utilizando os tipos de dados primitivos definidos no arquivo *types.h*. Em seguida, o código é apresentado na Imagem 10:

```
1  #ifndef STRING_H
2  #define STRING_H
3
4  /*A biblioteca types.h será incluída pois os tipos de
5  dados que serão usados aqui são declarados nela */
6
7  #include "types.h"
8
9  /*A função apresentada a seguir é utilizada verificar
10 o tamanho de uma string */
11
12 uint16 strlenh(string ch){
13     uint16 i = 1;
14     while(ch[i++]);
15     return --i;
16 }
17
18 #endif
```

Imagem 10: Código do arquivo *string.h*

2.1.3. Arquivo *system.h*

As funções descritas no arquivo *system.h* são usadas para fazer acesso direto às portas, para que seja possível manipular o cursor na tela. Neste arquivo também foi usado como biblioteca o arquivo *types.h*, para que fosse possível utilizar os tipos primitivos de dados. O código é apresentado em Imagem 11:

```
1  #ifndef SYSTEM_H
2  #define SYSTEM_H
3
4  /*A biblioteca types.h será incluída pois os tipos de dados que
5  serão usados aqui são declarados nela */
6
7  #include "types.h"
8
9  /*As funções abaixo servem para manipulação do cursor na tela. Por
10 isso devem fazer acesso direto às portas
11
12 O parametro passado será o valor da porta _port, então, a variável
13 rv se conectará a essa porta, e o valor contido na porta será
14 atribuído a essa variável.*/
15
16 uint8 inportb (uint16 _port){
17     uint8 rv;
18     __asm__ __volatile__ ("inb %1, %0"
19         : "=a" (rv)
20         : "dN" (_port));
21     return rv;
22 }
23
24 void outportb (uint16 _port, uint8 _data){
25     __asm__ __volatile__ ("outb %1, %0"
26         : "dN" (_port), "a" (_data));
27 }
28
29 #endif
```

Imagem 11: Código do arquivo *system.h*

2.1.4. Arquivo *screen.h*

As funções utilizadas no arquivo *screen.h* são funções previstas monitores VGA, que é o padrão utilizado para a tela de *boot* de qualquer sistema operacional. Nesse arquivo são adicionadas funções para limpar linhas, atualizar a localização do cursor na tela, fazer *scrollUp* da tela, limpar a tela, entre outras. Todas as funções foram feitas de acordo com o documento disponibilizado pelo professor para a orientação. Para o caso específico da função *printch(char)* foi utilizado a Tabela ACII. As funções presentes no arquivo serão explicadas por meio dos comentários presentes no código abaixo (devido ao tamanho do código, não foi possível gerar uma imagem do mesmo):

```
#ifndef SCREEN_H
#define SCREEN_H

#include "types.h"
#include "system.h"
#include "string.h"

int32 cursorX = 0, cursorY = 2;
const uint8 sw = 80, sh = 25, sd = 2;

void clearLine(uint8 from, uint8 to){

    /*A variável i é um auxiliar que define o ponto inicial em função da largura e
    profundidade (linha x coluna) de onde a função começará apagar a tela. Por
    exemplo, caso deseja-se apagar da linha 2 até a 4: A primeira posição no vetor
    vidmem a se pagar será a 320, e a posição final será a 800*/

    uint16 i = sw * from * sd;

    /*A linha abaixo inicializa a string vidmem, com a primeira posição de memória
    válida na tela*/

    string vidmem = (string)0xb8000;

    /*O loop abaixo percorre o vetor vidmem, onde a todas as posições acessadas
    pelo loop são limpas*/

    for(i; i < (sw * (to + 1) * sd); i++)
```

```
        vidmem[i] = 0x0;
    }
```

```
void updateCursor(){
    unsigned temp;

    /*Nessa variável será atribuído o valor da posição, em função dos eixos x e y*/

    temp = cursorY * sw + cursorX;

    /*Para a função outportb(), o primeiro parâmetro significa a porta onde se deseja
    escrever o dado, e o segundo parâmetro significa qual dado se deseja escrever na
    porta em questão*/

    /*Na linha abaixo, a posição é selecionada no registrador de controle*/

    outportb(0x3D4, 14);

    /*O byte de alta é enviado no barramento através da função abaixo*/

    outportb(0x3D5, temp >> 8);

    /*O byte de baixa é selecionado no registrador de controle*/

    outportb(0x3D4, 15);

    /*O byte de baixa da posição do cursor é enviado no barramento*/

    outportb(0x3D5, temp);
}
```

```
void clearScreen(){
```


/*A presente função limpa todas as linhas da tela, da primeira até a última. Ela faz esta limpeza através da função clearLine(), passando como parâmetro a primeira linha e a linha sh-1 (última). Depois desta limpeza, a função reposiciona o cursor do mouse para a primeira posição (0,0)*/

```
clearLine(0,sh-1);  
cursorX = 0;  
cursorY = 0;  
updateCursor();
```

```
}
```

```
void scrollUp(uint8 lineNumber){
```

/*A linha abaixo inicializa a string vidmem, com a primeira posição de memória válida na tela*/

```
string vidmem = (string)0xb8000;
```

```
uint16 i = 0;
```

/*É copiado para a primeira linha o conteúdo da lineNumber+1 linha abaixo da primeira linha. Ou seja, caso a função seja chamada como srollUp(5), o conteúdo da linha 6 será copiado para a primeira linha*/

```
for(i; i < sw * (sh - 1) * sd; i++)  
    vidmem[i] = vidmem[i + sw * sd * lineNumber];
```

/*as que foram copiadas são limpadas de sua posição antiga (original)*/

```
clearLine(sh - 1 - lineNumber, sh - 1);
```

/*O cursor é posicionado no ponto (0,0) caso deseja-se deslocar mais texto que existe na tela*/

```
if((cursorY - lineNumber) < 0){  
    cursorY = 0;
```

```

        cursorX = 0;
    }
    else
        cursorY -= lineNumber;

    /*Atualiza o cursor na tela*/
    updateCursor();
}

void newLineCheck(){

    /*A presente função faz um scrollUp() de uma linha, caso a última linha da tela
    seja preenchida, liberando uma nova linha*/

    if(cursorY >= sh - 1)
        scrollUp(1);
}

void printch(char c){

    /*A linha abaixo inicializa a string vidmem, com a primeira posição de memória
    válida na tela*/

    string vidmem = (string)0xb8000;

    switch(c){

        /*O caractere 0x08 corresponde ao Backspace, ou seja, para apagar um
        caractere. Caso seja a opção desejada, primeiro é feita uma verificação
        se existe caractere a ser apagado, e caso sim, o cursor volta uma
        posição no eixo x, e a string vidmem limpa o caractere onde estava
        o cursor no eixo x, antes de decrementar*/

        case (0x08):
            if(cursorX > 0){

```

```

        cursorX--;
        vidmem[(cursorY * sw + cursorX) * sd] = 0x00;
    }

```

/*O caractere 0x09 corresponde ao TAB. Sendo assim, quando entra-se no case em questão, o cursor se movimenta no eixo x em 8 posições*/
case (0x09):

```

        cursorX = (cursorX + 8) & ~(8 - 1);
        break;

```

/*O caractere em questão se trata do "carriage return", que reposiciona o cursor para o início da linha*/

```

case ('\r'):
    cursorX = 0;
    break;

```

/*O caractere em questão se trata do "new line", sendo assim, o cursor no eixo y é incrementado em um, passando para a linha abaixo, e o no eixo x, é reposicionado para o inicio da linha*/

```

case ('\n'):
    cursorX = 0;
    cursorY++;
    break;

```

/*Caso não seja nenhuma das opções anteriores, o caractere digitado é mostrado na tela com a cor branca, na posição atual do cursor. Quando o caractere é apresentado, o cursor no eixo x é incrementado em uma posição*/

```

default:
    vidmem[((cursorY * sw + cursorX)) * sd] = c;
    vidmem[((cursorY * sw + cursorX)) * sd + 1] = 0x0F;

```

```

        cursorX++;
        break;
    }

    /*esse if faz verificação se a linha já está cheia, e caso sim, posiciona
    o cursor na linha abaixo no eixo y, e na posição inicial do eixo x*/
    if (cursorX >= sw){
        cursorX = 0;
        cursorY++;
    }

    /*Com a função newLineCheck() verifica se a tela já está toda cheia*/
    newLineCheck();

    /*Atualiza a posição do cursor na tela*/
    updateCursor();
}

void print(string ch){

    /*A função em questão é usada para mostrar na tela uma string, fazendo um
    laço de repetição da posição 0 da string (primeiro caractere) até o último*/
    uint16 i = 0;

    for(i; i < strlen(ch); i++)
        printch(ch[i]);
}

#endif

```

2.2. Utilizando as funções em *kernel.c*

Para fazer o uso das funções construídas em *screen.h* no arquivo *kernel.c*, basta incluir a biblioteca no código C, e após isto, as funções estão prontas para serem chamadas. Alguns exemplos podem ser vistos na Imagem 12, Imagem 13 onde a função *clearScreen()* e *updateCursor()* são usadas respectivamente para limpar a tela logo após a frase “simpleOS” ser mostrada na tela e para atualizar a posição do cursor para a

posição desejada.

```
1 #include "include/screen.h"
2
3 kmain(){
4     char *vidmem = (char*) 0xb8000;
5     vidmem[0] = 's';
6     vidmem[1] = 0x02; /* cor do caracter: verde */
7
8     vidmem[2] = 'i';
9     vidmem[3] = 0x02;
10
11     vidmem[4] = 'm';
12     vidmem[5] = 0x02;
13
14     vidmem[6] = 'p';
15     vidmem[7] = 0x02;
16
17     vidmem[8] = 'l';
18     vidmem[9] = 0x02;
19
20     vidmem[10] = 'e';
21     vidmem[11] = 0x02;
22
23     vidmem[12] = '0';
24     vidmem[13] = 0x02;
25
26     vidmem[14] = 'S';
27     vidmem[15] = 0x02;
28
29     clearScreen();
30 }
```

Imagem 12: Código de *kernel.c* utilizando a função *clearScreen()*

```
1 #include "include/screen.h"
2
3 kmain(){
4
5     updateCursor();
6
7     char *vidmem = (char*) 0xb8000;
8     vidmem[0] = 's';
9     vidmem[1] = 0x02; /* cor do caracter: verde */
10
11     vidmem[2] = 'i';
12     vidmem[3] = 0x02;
13
14     vidmem[4] = 'm';
15     vidmem[5] = 0x02;
16
17     vidmem[6] = 'p';
18     vidmem[7] = 0x02;
19
20     vidmem[8] = 'l';
21     vidmem[9] = 0x02;
22
23     vidmem[10] = 'e';
24     vidmem[11] = 0x02;
25
26     vidmem[12] = '0';
27     vidmem[13] = 0x02;
28
29     vidmem[14] = 'S';
30     vidmem[15] = 0x02;
31
32 }
```

Imagem 13: Código de *kernel.c* utilizando a função *updateCursor()*

References

Castro, S. de (2018) “Instalando o Oracle VirtualBox no Ubuntu 14.04, 16.04 e 18.04”. Disponível em: <<https://www.blogopcaolinux.com.br/2016/05/instalando-virtualbox-no-ubuntu-lts.html>>. Acesso em: 19 set. 2018.